Beyond Accuracy

A Deep Dive into AU-PRC & AUC-ROC For Imbalanced Classification

42.515 - MLA
MTD DS Term 2/ 2026
Team 02

Emmanuel 1011048
GunHyun 1010966
Ian 1010766
Nicole 1011069

Table of Contents

Your Model is 95% Accurate
It's Also Useless

- Hospital deploys a pneumonia risk model
- Model flags asthma patients as low risk:
  they always go to ICU and survive
- Accurate on paper. Dangerously wrong in practice.

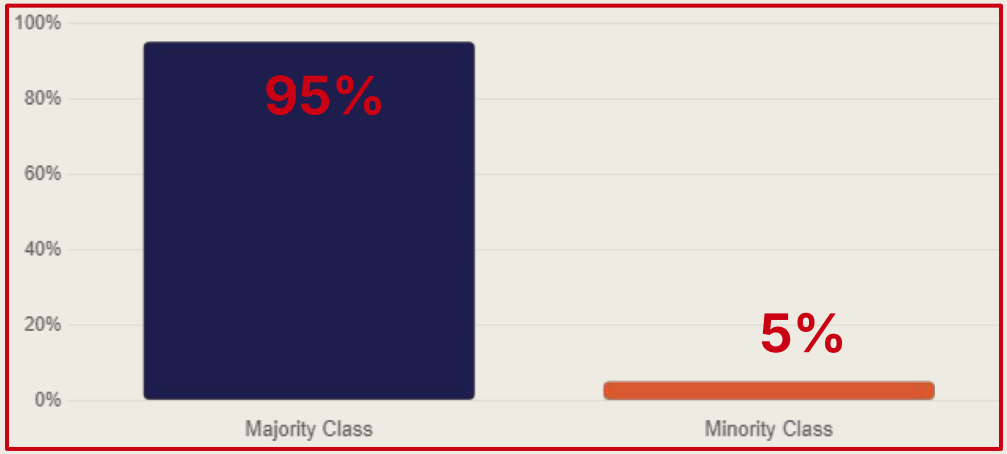
When One Class Drowns Out the Other

💳 Fraud Detection

95% legitimate transactions
5% fraudulent

🏥 Medical Diagnosis

95% healthy patients
5% have the disease



The Model That Learned Nothing

Dataset

950 Healthy
50 Sick

Accuracy

95% ✓

Sick Caught

0 ×

High accuracy. Zero value. The model simply ignored the minority class entirely.

Not All Mistakes Are Equal

False Positive

Flagging a healthy person as sick
→ Extra tests, minor inconvenience

False Negative ⚠

Missing a sick person entirely
→ No treatment. Serious harm.

🏥 **Medical**

Missed diagnosis

💳 **Fraud**

Undetected transaction

🏭 **Manufacturing**

Defective product shipped

We Need Better Metrics.

01. Accuracy is broken for imbalanced data

02. We need metrics focused on the minority class

03. Enter: AUC-ROC and AU-PRC

The Confusion Matrix

Everything Start Here

	Predicted Positive	Predicted Negative
Actually Positive	True Positive (TP) Caught it!	False Negative (FN) Missed it
Actually Negative	False Positive (FP) False alarm	True Negative (TN) Correctly ignored

Every metric you will see today is built from just these four numbers

Accuracy adds them all up and divides the total

Precision - When I raise an alarm,
Am I usually right?

$$Precision = \frac{TP}{TP + FP}$$

My fraud model raises 600 alerts.
400 are real fraud.
200 are legitimate transactions wrongly flagged.

Precision - When I raise an alarm,
Am I usually right?

$$Precision = \frac{TP}{TP + FP}$$

My fraud model raises 600 alerts.
400 are real fraud.
200 are legitimate transactions wrongly flagged.

$$Precision = \frac{400}{400 + 200} = \frac{400}{600} = 0.667$$

Precision - When I raise an alarm,
Am I usually right?

$$Precision = \frac{TP}{TP + FP}$$

My fraud model raises 600 alerts.
400 are real fraud.
200 are legitimate transactions wrongly flagged.

$$Precision = \frac{400}{400 + 200} = \frac{400}{600} = 0.667$$

67% of my alerts are correct.
1 in 3 is a false alarm.

Low precision = too many false alarms.
People stop trusting the model.

Recall - Of all the real cases,
How many did I catch?

$$Recall = \frac{TP}{TP + FN}$$

There were 500 fraudulent transactions in total.
My model caught 400.
It missed 100.

Recall - Of all the real cases,
How many did I catch?

$$Recall = \frac{TP}{TP + FN}$$

There were 500 fraudulent transactions in total.
My model caught 400.
It missed 100.

$$Recall = \frac{400}{400 + 100} = \frac{400}{500} = 0.800$$

Recall - Of all the real cases,
How many did I catch?

$$Recall = \frac{TP}{TP + FN}$$

There were 500 fraudulent transactions in total.
My model caught 400.
It missed 100.

$$Recall = \frac{400}{400 + 100} = \frac{400}{500} = 0.800$$

80% of real fraud was caught.
But 100 fraudulent transactions slipped through undetected.

Low recall = dangerous misses.
The cases that matter most are slipping through.

The Tension Between
Precision & Recall

Strategy	Recall	Precision
Flag every single transaction as fraud	100% Miss nothing	Very low Endless false alarms
Only flag when you are 99% certain	Very low Miss almost everything	High Almost no false alarms
Balanced threshold	Moderate	Moderate

The Tension Between Prec[illegible]call

Strategy	Recall	Precision
Flag every single transaction as fra[illegible]	1[illegible]0%	Ver[illegible]low
Only flag when you are 99% certain	Very low	[illegible] alarms
Balanced threshold	Miss almost everyt[illegible]	High
	Moderate	Almost no false alarms
		[illegible]ate

There is always a trade-off.

The F1 score is the standard way to combine both into one number

The Tension Between
Precision & Recall

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

Why harmonic mean and not arithmetic mean?

Model that flags every single patient as sick:

- Recall = 1.0
  catches everyone
- Precision = 0.05
  only 5% of the population is actually sick

- Arithmetic mean = 0.525 sounds acceptable
- F1 = 0.095 correctly terrible

The harmonic mean is zero if either value is zero.

It only gives a high score when both Precision & Recall are genuinely high

Your Model Does Not Say "Fraud", it gives a Score

Transaction	Model Score	Threshold =0.8	Threshold =0.5	Threshold =0.2
Transaction A	0.92	Flagged	Flagged	Flagged
Transaction B	0.61	Not Flagged	Flagged	Flagged
Transaction C	0.18	Not Flagged	Not Flagged	Flagged

Every model outputs a probability score between 0 and 1. You choose a threshold τ. Any score above the threshold is flagged as positive.

Every time you move the threshold, you get a different Precision and a different Recall.

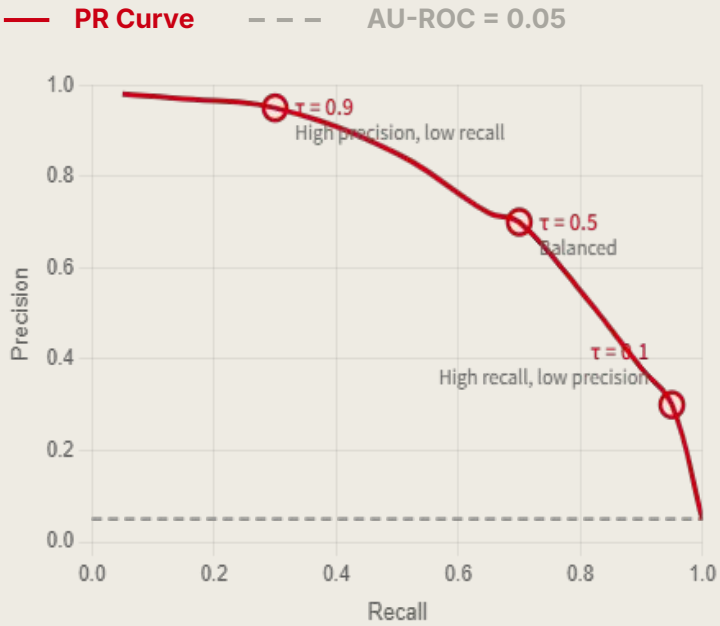
There is no single "correct" threshold. It depends on what matters more in your application.

The Precision-Recall Curve plots all possible thresholds at once so you can see the full picture.

Plot Every Threshold

The Precision - Recall Curve

Threshold (τ)	Recall	Precision	What this means
τ = 0.9 (high)	0.30	0.95	Very selective - only the most certain cases flagged, very few false alarms, but missing a lot
τ = 0.5 (mid)	0.70	0.70	Balanced - catching more but also more false alarms
τ = 0.1 (low)	0.95	0.30	Catches almost everything - but 70% of alerts are wrong



Plot Every Threshold

The Precision - Recall Curve

Threshold (τ)	Recall	Precision	What this means
τ = 0.9 (high)	0.30	0.95	Very selective - only the most certain cases flagged, very few false alarms, but missing a lot
τ = 0.5 (mid)	0.70	0.70	Balanced - catching more but also more false alarms
τ = 0.1 (low)	0.95	0.30	Catches almost everything - but 70% of alerts are wrong

$$AU-PRC = \sum_{n=1}^{N} \left(R_n - R_{n-1} \right) \times P_n$$

The area under this curve summarises performance across every possible threshold in one number. Closer to 1.0 is better.

The random baseline for AU-PRC is NOT 0.5. It equals the class prevalence.

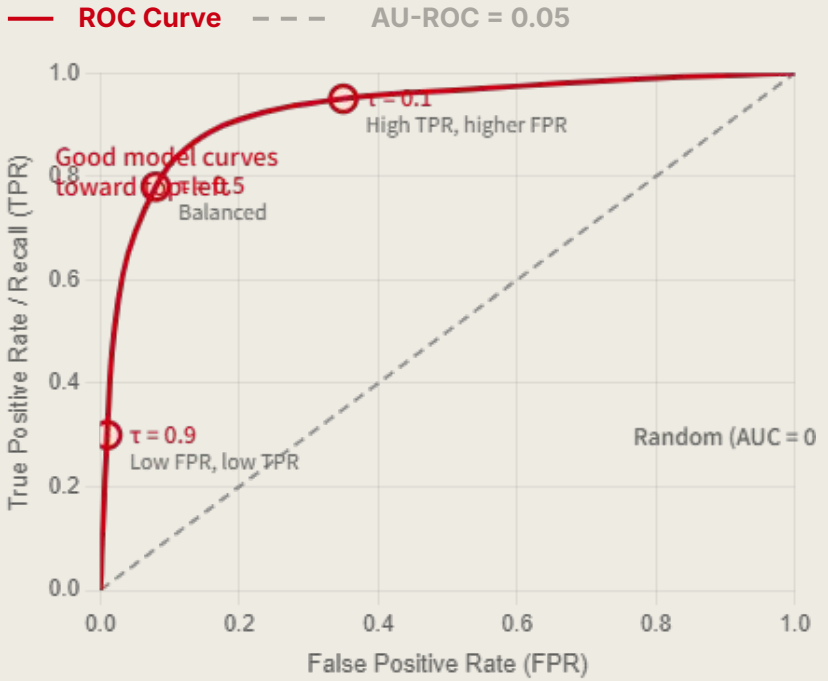
If only 5% of your data is positive, a random model scores AU-PRC ≈ 0.05.

This means a score of 0.70 on an imbalanced dataset is actually very good.

The ROC Curve

A Different View of the Same Threshold Sweep

Threshold (τ)	TPR (Recall)	FPR	What this means
τ = 0.9 (high)	0.30	0.01	Catches very little, but almost no false alarms
τ = 0.5 (mid)	0.70	0.02	Balanced
τ = 0.1 (low)	0.95	0.08	Catches almost everything more false alarms



The ROC Curve

A Different View of the Same Threshold Sweep

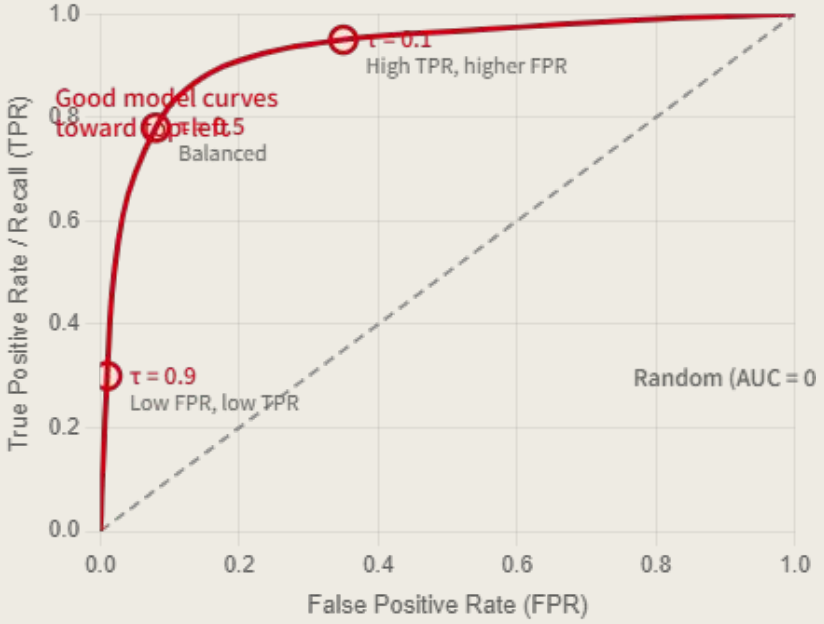
$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

The diagonal line represents a random model

AUC-ROC = 0.5. A good model curves toward the top-left corner.

AUC-ROC = 1.0 is perfect.



AUC-ROC interpretation:

The probability that a randomly chosen positive example is assigned a higher score by the model than a randomly chosen negative example.
(Hanley & McNeil, 1982)

Why AUC-ROC Lies

When Your Data is Imbalanced

10,000 patients. 500 have the disease. 9,500 are healthy. The model raises 600 alerts. 400 are correct. 200 are wrong. 100 sick patients were missed.

Why AUC-ROC Lies

When Your Data is Imbalanced

10,000 patients. 500 have the disease. 9,500 are healthy. The model raises 600 alerts. 400 are correct. 200 are wrong. 100 sick patients were missed.

$$FPR = \frac{FP}{FP + TN} = \frac{200}{200 + 9300} = \frac{200}{9500} = 0.021$$

$$Precision = \frac{TP}{TP + FP} = \frac{400}{400 + 200} = \frac{400}{600} = 0.667$$

$$Recall = \frac{TP}{TP + FN} = \frac{400}{400 + 100} = \frac{400}{500} = 0.800$$

Metric	Value	What it tells you
FPR	0.021 (2.1%)	Looks great ROC curve sits near the top-left
Precision	0.667 (66.7%)	1 in 3 alerts is wrong
Recall	0.800 (80.0%)	Still missing 100 sick patients

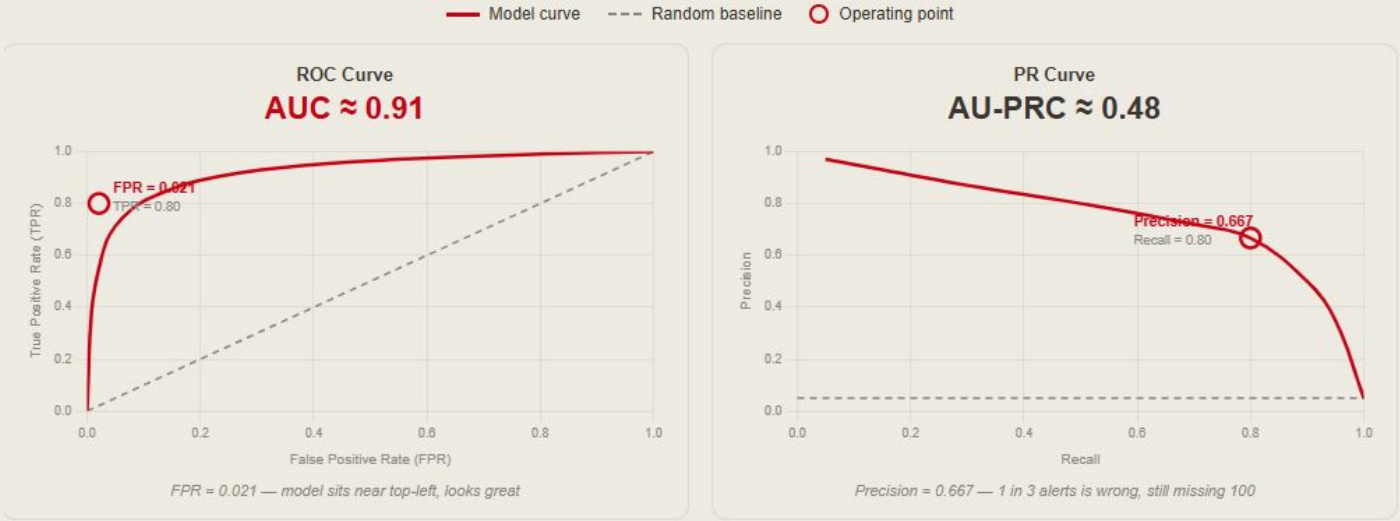
The Root Cause: TN=9,300 is sitting in the denominator of FPR. Even though we generated 200 false alarms, dividing 9,500 makes FPR look tiny.

AUC-ROC gets credit for correctly ignoring 9,300 healthy people.
AU-PRC does not becuase it never sees TN at all

Why AUC-ROC Lies

When Your Data is Imbalanced

10,000 patients. 500 have the disease. 9,500 are healthy. The model raises 600 alerts. 400 are correct. 200 are wrong. 100 sick patients were missed.

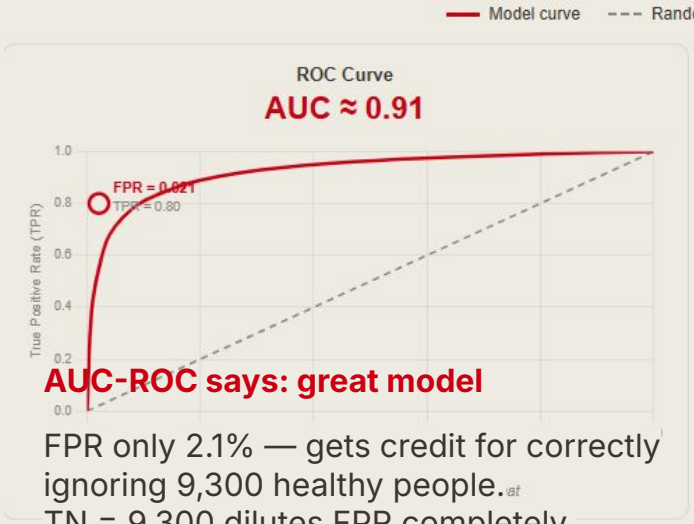


Metric	Value
FPR	0.021 (2.1%)
Precision	0.667 (66.7%)
Recall	0.800 (80.0%)

Why AUC-ROC Lies

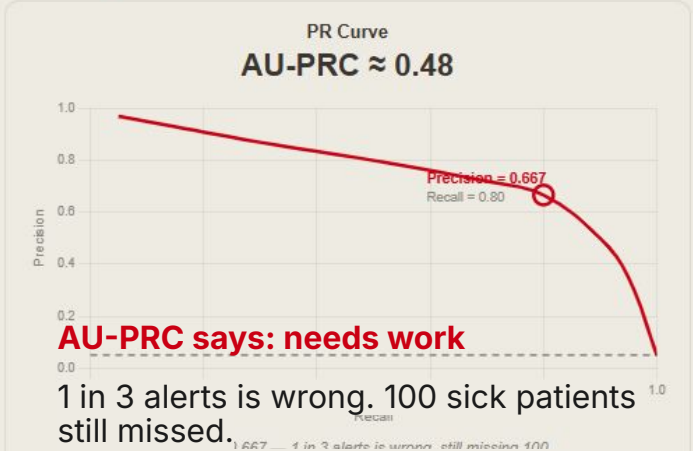
When Your Data is Imbalanced

10,000 patients. 500 have the disease. 9,500 are healthy. The model raises 600 alerts. 400 are correct. 200 are wrong. 100 sick patients were missed.



AUC-ROC says: great model

FPR only 2.1% — gets credit for correctly ignoring 9,300 healthy people.
TN = 9,300 dilutes FPR completely.



AU-PRC says: needs work

1 in 3 alerts is wrong. 100 sick patients still missed.
TN is invisible — no credit for the easy task.

Metric	Value
FPR	0.021 (2.1%)
Precision	0.667 (66.7%)
Recall	0.800 (80.0%)

Why AUC-ROC Lies

When Your Data is Imbalanced

$$FPR = \frac{FP}{FP + TN}$$

TN dilutes this

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

No TN anywhere

- AUC-ROC is optimistic under imbalance because it rewards the model for handling the easy majority class.
- AU-PRC strips that credit away entirely.

Why AUC-ROC Lies

When Your Data is Imbalanced

$$FPR = \frac{FP}{FP + TN}$$

TN dilutes this

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

No TN anywhere

- AUC-ROC is optimistic under imbalance because it rewards the model for handling the easy majority class.
- AU-PRC strips that credit away entirely.

So what does this really mean?

AUC-ROC VS AU-PRC

Which One Should You Use?

SIMPLIFIED

Presume, your job is to find 5 sick people hiding in a crowd of 95 healthy people. 100 in total.

You build an AI to help you. The AI looks at everyone and raises an alarm when it thinks someone is sick.

THE PROBLEM

The AI learns a cheat.
Most people are healthy, so it stays quiet and avoids mistakes on the 95 healthy people.

AUC-ROC rewards it for this. But leaving healthy people alone is easy.
We hired it to find the 5 sick people.

AU-PRC only asks two things:

1. Of the 5 sick people, how many did you find?
2. Of every alarm raised, how many were actually sick?

AUC-ROC gives credit for the easy task

AU-PRC only scores the hard one.

AUC-ROC VS AU-PRC

Before the summary - A Question for You

THE PROBLEM

Same crowd. 100 people. 5 are sick. 95 are healthy.

The AI raises 20 alarms.
Of those 20 alarms, only 4 are actually sick people.
The AI missed 1 sick person entirely.

Answer these 4 questions:

1. What is the Precision?
2. What is the Recall?
3. What is the FPR?
4. Which metric makes this AI look better than it really is, AUC-ROC or AU-PRC? Why?

AUC-ROC VS AU-PRC

Before the summary - A Question for You

THE PROBLEM

Same crowd. 100 people. 5 are sick. 95 are healthy.

The AI raises 20 alarms.
Of those 20 alarms, only 4 are actually sick people.
The AI missed 1 sick person entirely.

Metric	Value
TP	4
FP	16
FN	1
TN	79

Metric	Value	
Precision	4/20=0.20	1 in 5 alarm correct
Recall	⅘=0.80	Caught 4 out of 5 people
FPR	16/95=0.168	Flagged 17% of healthy people

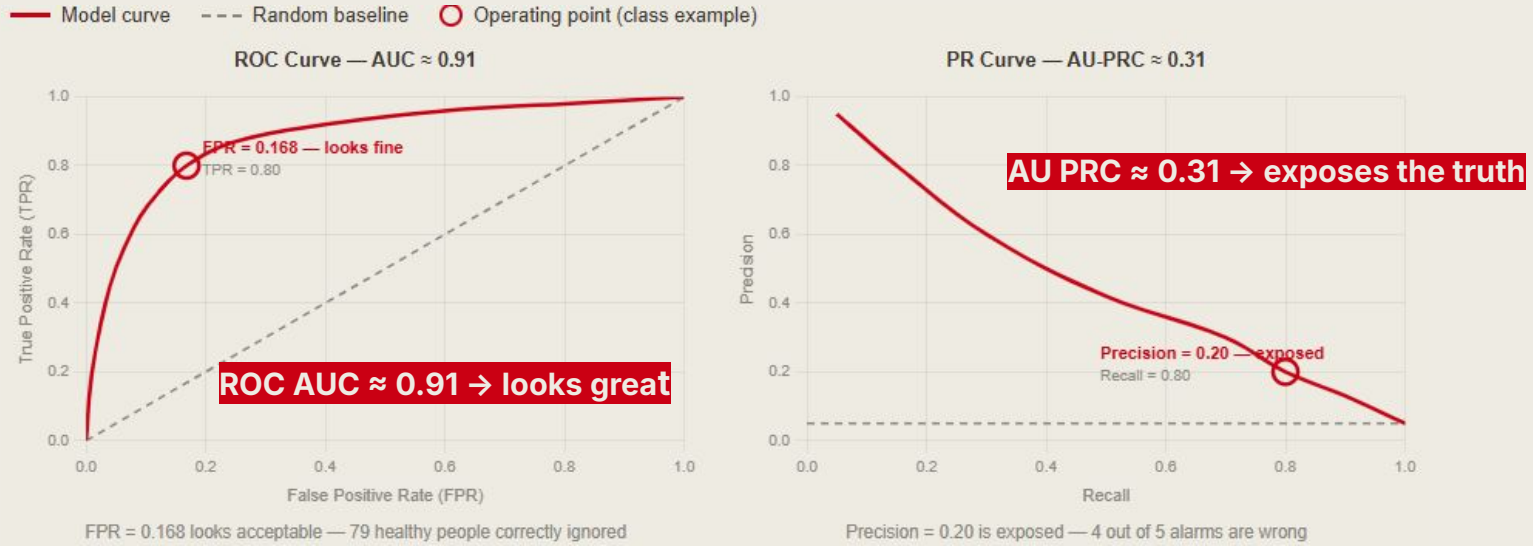
AUC-ROC VS AU-PRC

Before the summary - A Question for You

THE PROBLEM

Same crowd. 100 people. 5 are sick. 95 are healthy.

The AI raises 20 alarms.
Of those 20 alarms, only 4 are actually sick people.
The AI missed 1 sick person entirely.



FPR = 0.168 looks acceptable — 79 healthy people correctly ignored

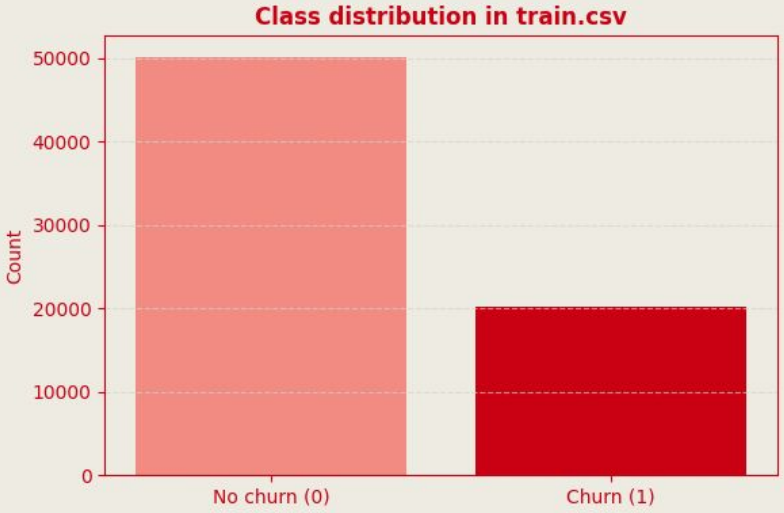
Precision = 0.20 is exposed — 4 out of 5 alarms are wrong

AUC-ROC VS AU-PRC

Which One Should You Use?

	AUC-ROC	AU-PRC
What it measures?	Can the model rank positives above negatives?	How precise is the model when it flags positives?
Uses TN?	Yes - in FPR denominator	No - TN never appears
Random baseline	0.5 always	Equals class prevalence (e.g. 0.05 at 5% imbalance)
Behaviour under imbalance	Inflated - looks too good	Accurate - shows the real struggle
Best used when	Classes are roughly balanced	Minority class is what matters
Examples	Gender classification, balanced test sets	Fraud detection, disease diagnosis, churn prediction

Dataset Overview



The target variable is **ChurnStatus**.

The dataset is **moderately imbalanced**.

Class distribution:

- **No churn: 50,155 (71.2%)**
- **Churn: 20,275 (28.8%)**

This means the positive class is less frequent, so model evaluation should go beyond accuracy alone.

Data Audit / Missing Value

```
df = train_df.copy()
df[target_col] = df[target_col].map(
  {"No": 0, "Yes": 1})

print("Target distribution:")
display(df[target_col].value_counts().rename(
  index={0: "No churn", 1: "Churn"}))

print("Target proportions:")
display(df[target_col].value_counts(
  normalize=True).rename(
  index={0: "No churn", 1: "Churn"}))

missing_df = pd.DataFrame({
    "missing_count": df.isna().sum(),
    "missing_share": df.isna().mean()
}).sort_values("missing_share", ascending=False)
```

A quick audit was performed to inspect target balance, column types, and missing values.

The highest missingness appears in:

- **Offer: 68.47%**
- **InternetType: 24.16%**
- **PrioritySupport: 17.58%**
- **DigitalInvoicing: 17.28%**

The preprocessing pipeline is used to handle mixed feature types and missing values consistently across models.

Feature & Split Setup

```
feature_df = df.drop(columns=[target_col])
if id_col in feature_df.columns:
    feature_df = feature_df.drop(columns=[id_col])

X = feature_df.copy()
y = df[target_col].copy()

numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = X.select_dtypes(include=["object"]).columns.tolist()

numeric_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer([
    ("num", numeric_transformer, numeric_features),
    ("cat", categorical_transformer, categorical_features)
])

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=RANDOM_STATE
)
```

CustomerID was removed as an identifier and **ChurnStatus** was used as the target.

A **stratified train-validation split** was applied to preserve class proportions.

Split summary:

- **Train shape: (52,822, 43)**
- **Validation shape: (17,608, 43)**
- **Train positive rate: 0.2879**
- **Validation positive rate: 0.2879**

Stratification ensures both sets reflect the same class imbalance as the original dataset.

Models Used

```
available_models = {}

# Logistic Regression is always included
available_models["Logistic Regression"] = LogisticRegression(
    max_iter=2000,
    random_state=RANDOM_STATE
)

# LightGBM
try:
    from lightgbm import LGBMClassifier
    available_models["LightGBM"] = LGBMClassifier(
        n_estimators=300,
        learning_rate=0.05,
        num_leaves=31,
        random_state=RANDOM_STATE,
        verbose=-1
    )
    print("LightGBM loaded successfully.")
except Exception as e:
    print("LightGBM not available:", e)

# XGBoost
try:
    from xgboost import XGBClassifier
    available_models["XGBoost"] = XGBClassifier(
        n_estimators=300,
        learning_rate=0.05,
        max_depth=6,
        subsample=0.9,
        colsample_bytree=0.9,
        eval_metric="logloss",
        random_state=RANDOM_STATE
    )
    print("XGBoost loaded successfully.")
except Exception as e:
    print("XGBoost not available:", e)
```

Three models were compared:

- **Logistic Regression**
- **LightGBM**
- **XGBoost**

Logistic Regression provides a strong linear baseline.

LightGBM and XGBoost were included as more powerful tree-based ensemble methods.

Evaluation Metrics

```python
def evaluate_model(model_name, model, X_train, X_valid, y_train, y_valid):
    pipe = build_pipeline(model)
    pipe.fit(X_train, y_train)

    y_score = pipe.predict_proba(X_valid)[:, 1]
    y_pred = pipe.predict(X_valid)

    fpr, tpr, _ = roc_curve(y_valid, y_score)
    precision, recall, _ = precision_recall_curve(y_valid, y_score)

    roc_auc = roc_auc_score(y_valid, y_score)
    avg_precision = average_precision_score(y_valid, y_score)

    metrics_row = {
        "model": model_name,
        "roc_auc": roc_auc,
        "avg_precision": avg_precision,
        "accuracy": (y_pred == y_valid).mean(),
        "positive_rate_valid": y_valid.mean()
    }

    return {
        "model_name": model_name,
        "pipeline": pipe,
        "y_pred": y_pred,
        "y_score": y_score,
        "fpr": fpr,
        "tpr": tpr,
        "precision": precision,
        "recall": recall,
        "roc_auc": roc_auc,
        "avg_precision": avg_precision,
        "confusion_matrix": confusion_matrix(y_valid, y_pred),
        "report_df": pd.DataFrame(classification_report(y_valid, y_pred, output_dict=True)).transpose(),
        "metrics_row": metrics_row
    }
```

Each model was evaluated using:

- **ROC-AUC**
- **Average Precision (AU-PRC style evaluation)**
- **Confusion matrix**

ROC-AUC measures how well the model separates classes overall.

Average Precision focuses more directly on **positive-class performance**.

This is important in churn prediction because the minority class is the class we care most about detecting.

Results on the Original Dataset

Model	ROC_AUC	avg_precision
LightGBM	**0.910634**	**0.852094**
XGBoost	0.907935	0.846332
Logistic Regression	0.899270	0.828635

On the original dataset, all models achieved **high ROC-AUC and strong Average Precision**.

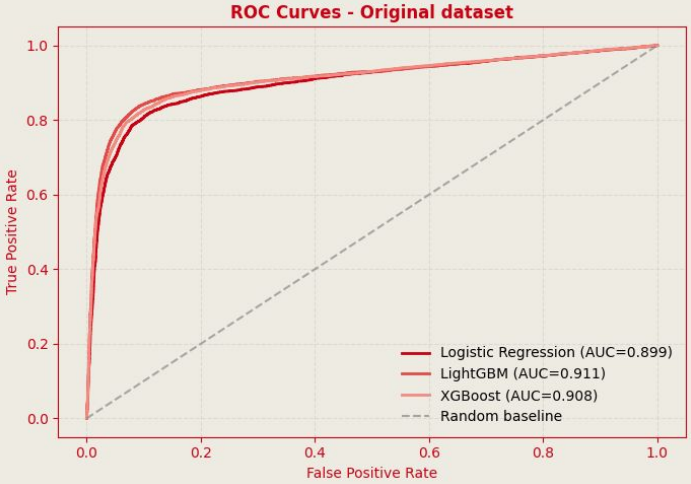
Results:

- **LightGBM:**
  ROC-AUC = **0.9106**, Average Precision = **0.8521**
- **XGBoost:**
  ROC-AUC = **0.9079**, Average Precision = **0.8463**
- **Logistic Regression:**
  ROC-AUC = **0.8993**, Average Precision = **0.8286**

Among the three models, **LightGBM performed best** on both metrics.

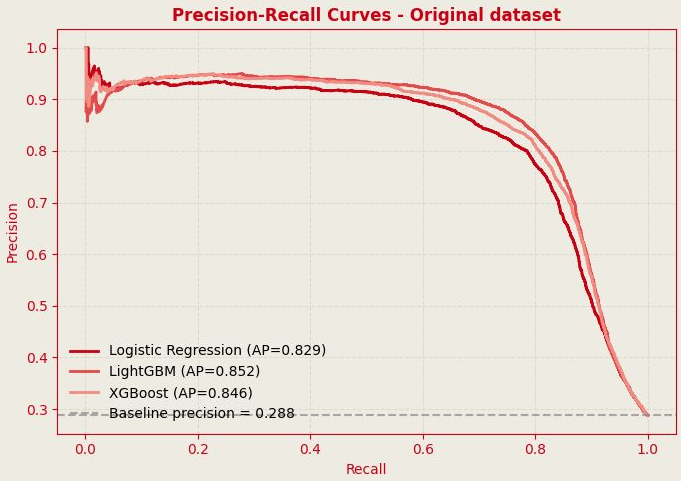
ROC & PR Curves

The Original Dataset



Model	ROC (Original)
LightGBM	0.911
XGBoost	0.908
Logistic Regression	0.899

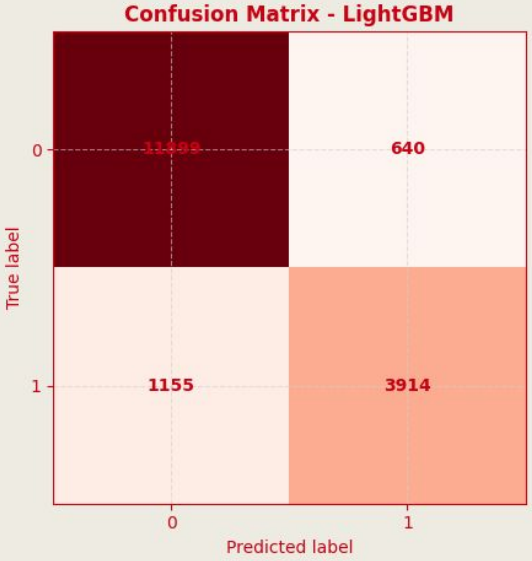
Model	AU-PRC (Original)
LightGBM	0.852
XGBoost	0.846
Logistic Regression	0.828

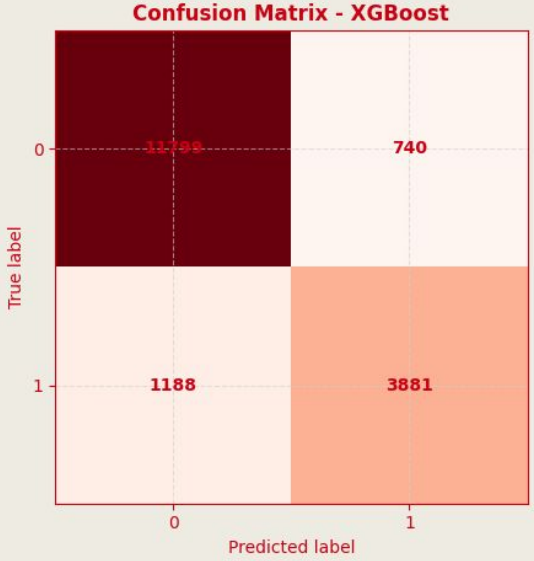


On the original dataset, the ROC curves are all strong and close together.
The PR curves also show good performance, with **LightGBM slightly ahead**.
At this imbalance level, both ROC-AUC and AU-PRC suggest that the models perform well.

However, the difference between the two metrics is not very dramatic as the dataset is not extremely imbalanced.

Confusion Matrix Results






```
ConfusionMatrixDisplay(
  res["confusion_matrix"]).plot()
plt.title(f'Confusion Matrix - {res["model_name"]}')
```

The confusion matrices provide a class-level view of performance.

Minority-class recall for churn was:

- **Logistic Regression: 0.750**
- **LightGBM: 0.772**
- **XGBoost: 0.766**

LightGBM produced the strongest balance between precision and recall for the churn class.

This supports the metric table, where **LightGBM also had the best ROC-AUC and Average Precision.**

The Dataset's Existing Imbalance

Original Churn Dataset

Class distribution is already imbalanced — approximately 71% no-churn vs 29% churn — but not extreme enough to make AUC-ROC look obviously misleading.

The Problem with Moderate Imbalance

Models can still achieve good AUC-ROC on a moderately imbalanced dataset, masking weaker performance on the minority class that AU-PRC would expose.

Our Approach

We create a more imbalanced version of the same dataset by downsampling the positive (churn) class to a 10% prevalence rate. This keeps results grounded in real data while amplifying the metric divergence.

Original Class Split

71% No Churn

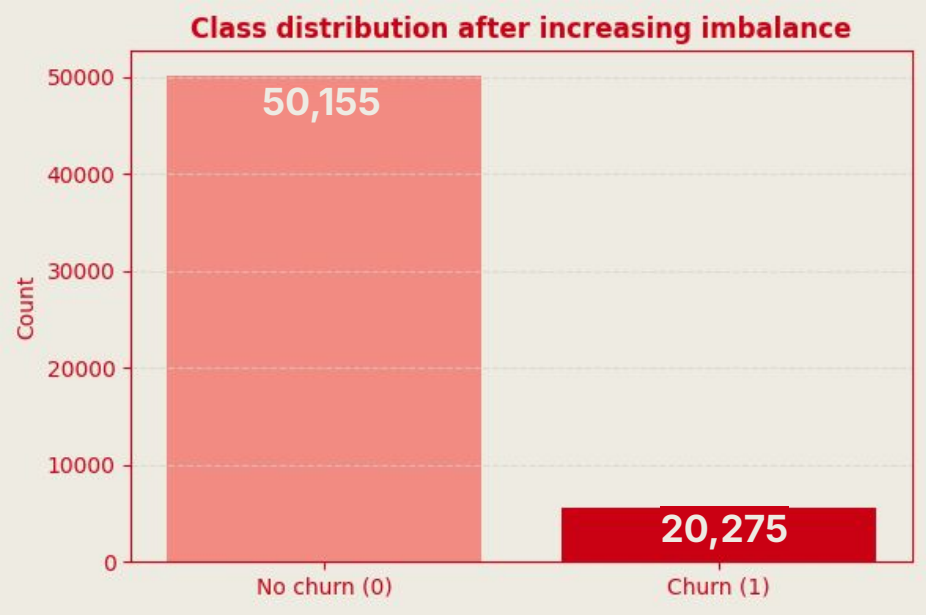
Downsampled Target Split

90% No Churn

Goal: observe how each metric responds when the positive class becomes rarer — all else equal.

Amplifying the Imbalance

Downsampling to 10% Positive Rate



What We Did

1 **Keep all negatives (No Churn)**

All 50,000+ no-churn records are retained unchanged.

2 **Downsample positives (Churn)**

Churners are randomly sampled down to ~10% of dataset size.

3 **Re-split & re-train**

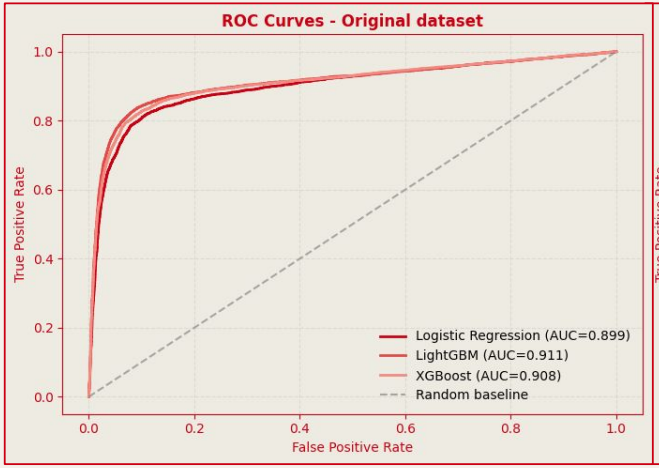
Same 75/25 stratified train-validation split, same models, same pipeline.

4 **Re-evaluate metrics**

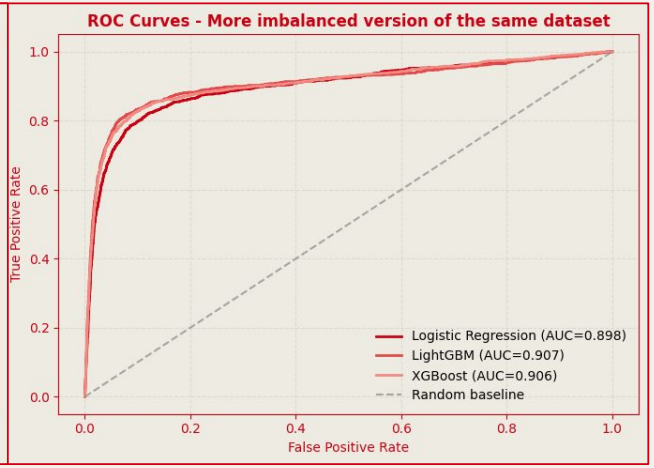
Both AUC-ROC and AU-PRC are recomputed under identical conditions.

AUC-ROC

Barely Moves Under Severe Imbalance



Original Dataset (29% positive)

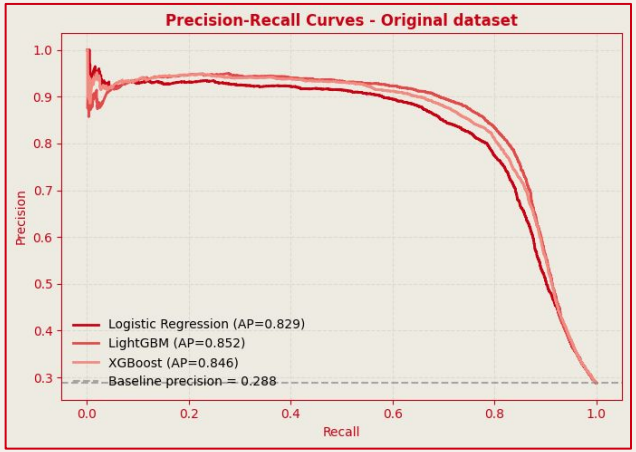


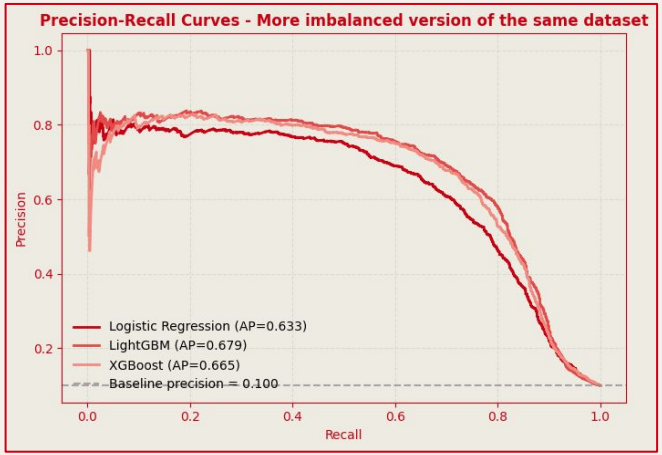
More Imbalanced (10% positive)

Model	ROC Original	ROC Imbalanced	Δ Change
LightGBM	0.911	0.907	**−0.004**
XGBoost	0.908	0.906	**−0.002**
Logistic Regression	0.899	0.898	**−0.001**

AUC-ROC - Drops Sharply

Revealing the True Struggle





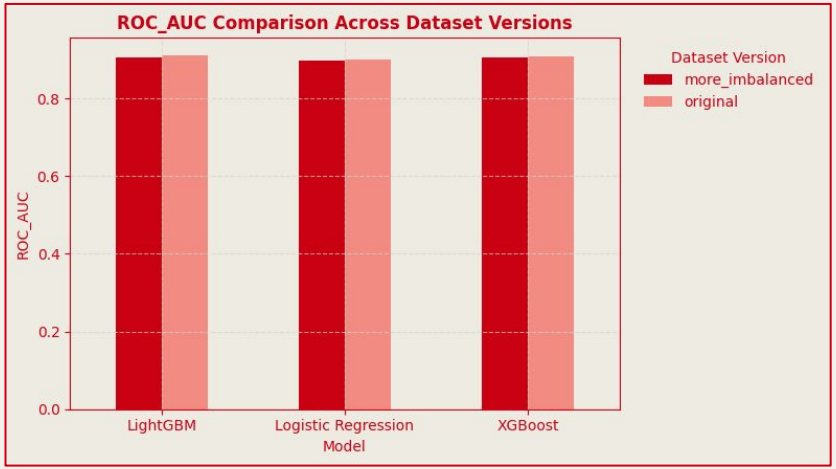
Original Dataset (AP baseline = 0.288)

More Imbalanced (AP baseline = 0.100)

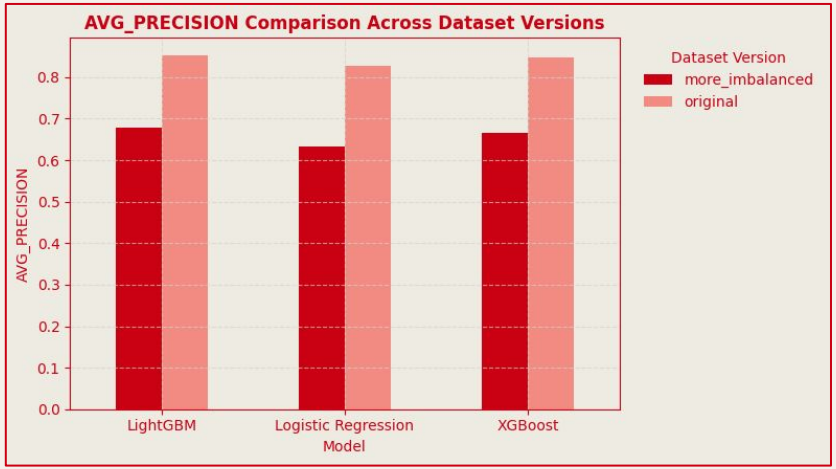
Model	AU-PRC (Original)	AU-PRC (Imbalanced)	Δ Change
LightGBM	0.852	0.679	**−0.173**
XGBoost	0.846	0.665	**−0.181**
Logistic Regression	0.828	0.633	**−0.195**

Metric Comparison

AUC-ROC vs AU-PRC Across Dataset Versions



AUC-ROC — nearly identical bars across versions



AU-PRC — orange (original) clearly higher than blue (imbalanced)

Key Insight: **The left chart (AUC-ROC) shows bars of nearly equal height between the original and imbalanced datasets, the metric is desensitised to the change. The right chart (AU-PRC) shows a dramatic drop in the bars: AU-PRC correctly registers that the model's ability to identify churners has degraded significantly when churners become rarer.**

What the Experiment Shows

1. **ROC-AUC stays high** on imbalanced data because **FPR is diluted by the large negative class**.

2. **AU-PRC drops sharply when positives become rarer**.
It captures real-world model utility better.

3. In this experiment: **ROC-AUC fell < 1 pt**
Average Precision fell 17–20 pts when **imbalance increased**.

4. For churn, fraud, medical diagnosis,
where catching the **positive class matters, AU-PRC is preferred**.

For imbalanced classification:
AU-PRC is often a better reflection of practical model usefulness than ROC-AUC.